

1. But du TP :

Le but de ce TP est de se familiariser avec l'utilisation pratique de la modélisation et de la résolution de problèmes de satisfaction de contraintes (CSP). Dans ce TP, nous allons utiliser le langage Python et l'outil « Google OR-Tools », avec la bibliothèque « cp_model » et le solveur « CP-SAT ».

2. OR-Tools

OR-Tools (<https://developers.google.com/optimization>) est une suite logicielle open source d'optimisation, conçue pour résoudre les problèmes qualifiés difficiles avec la programmation en nombres entiers (IP) et linéaires (PL) et la programmation par contraintes (CSP). OR-Tools est créé en C++, mais il est également utilisable avec Python, Java ou C# (sur la plate-forme .NET).

3. Installation

Installez OR-Tools à l'aide de la commande suivante :

```
python -m pip install ortools ou  
pip install ortools
```

Si vous utilisez un IDE évolué comme « PyCharm », ou « Visual Studio Code », « jupyter notebook », etc, vous pouvez installer OR-Tools à partir de l'interface du gestionnaire des packages intégré à l'IDE.

Testez votre installation :

```
python -c "import ortools; print(ortools.__version__)"
```

NB : Sous Windows il faut avoir « Microsoft Visual C++ Redistributable » pour visual Studio 2019 ou 2022 : <https://visualstudio.microsoft.com/fr/downloads/?q=Visual+C%2B%2B+Redistributable+for+Visual+Studio>

1. Un simple CSP

L'exemple suivant utilise l'outil OR-Tools pour résoudre le CSP à contrainte linéaires suivant :

$CSP1 = (X, D, C)$,

$X = \{x, y\}$,

$D(x) = [0, 5]$, $D(y) = \{2, 3, 8\}$

$C = \{x + y < 5, x \neq y\}$

Tester le code suivant:

```
# import cp_model library
from ortools.sat.python import cp_model

# création d'un modèle.
model = cp_model.CpModel()

# Déclaration de la variable x, elle prend des valeurs entre 0 et 5
x = model.NewIntVar(0, 5, 'x')
# Déclaration de la variable y, elle prend les valeurs 2,3 ou 8
y = model.NewIntVarFromDomain(cp_model.Domain.FromValues([2, 3, 8]), 'y')

# déclaration des contraintes:
model.Add(x + y < 5)
model.Add(x != y)

#déclaration du solveur et recherche des solutions
solver = cp_model.CpSolver()
status = solver.Solve(model)

#affichage d'une solution si elle existe.
if status == cp_model.OPTIMAL or status == cp_model.FEASIBLE:
    print('x = %i' % solver.Value(x))
    print('y = %i' % solver.Value(y))

else:
    print('No solution found.')
```

L'exemple précédent affiche une seule solution, on peut afficher toutes les solutions possibles de la manière suivante :

```
if status == cp_model.OPTIMAL or status == cp_model.FEASIBLE:

# Search for all solutions
solver.parameters.enumerate_all_solutions = True
solver.Solve(model, cp_model.VarArraySolutionPrinter([x, y]))
else:
    print('No solution found.')
```

Exercices

Exercice 01:

Soit le CSP suivant :

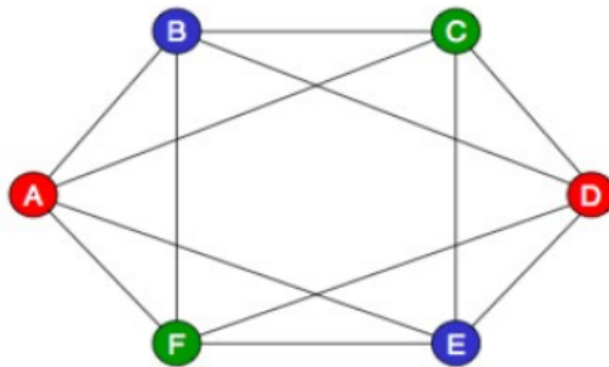
$X = \{x, y, z\}$, $D(x) = [0, 10]$, $D(y) = \{2, 3, 8\}$, $D(z) = [1, 8]$

$C = \{x + y == 8, x > y, z + y \leq x\}$

- Écrivez le programme Python qui résout ce CSP.
- Vérifiez si les solutions sont correctes.

Exercice 02 :

Écrire à l'aide de la bibliothèque « cp_model » de l'outil « OR-Tools » le programme Python qui résout le problème de coloriage de graphe suivant :



Exercice 03:

En utilisant la bibliothèque « OR-tools » écrivez un programme pour résoudre le Puzzles Crypto-Arithmétique : **TWO+TWO=FOUR**

Affichez toutes les solutions possibles.

NB: pour exprimer la contrainte impliquant que les variables X, Y, Z prennent des valeurs différentes en utilise la contrainte : `model.AddAllDifferent([X, Y, Z])`

- Vérifiez si les solutions sont correctes.

Exercice 04:

Écrire le programme CSP à l'aide de l'outil « OR-Tools » qui donne l'âge de chacun des cinq personnes, H, K, L, O et Y en respectant les contraintes suivantes :

- les âges varient entre onze et vingt-quatre ans ;
- Les personnes H, K, O et Y ont des âges différents ;
- K est moins de huit ans plus jeune que H, qui lui n'a pas le même âge que Y ;
- Y est plus jeune que L, et L a le même âge que H ;
- O est plus de dix ans plus jeune que H ;
- La différence d'âge entre O et H est impaire ;

NB: Expression de la contrainte modulo : si on veut tester si $X \% 3 = 1$ on ajoute la contrainte:

`model.AddModuloEquality(1, X, 3)`

Pour résoudre l'exercice, il est possible de déclarer des variables intermédiaires.

Exercice 05 :

On pose 6 tasses retournées côte à côte sur une même ligne. Chaque tasse recouvre exactement une petite balle. Les tasses sont numérotées consécutivement de 1 à 6. Chaque balle est peinte d'une couleur différente : verte, bleu , orange, violet, rouge et jaune. Vous savez que :

1. La balle bleue est cachée sous la tasse 1.
2. La balle verte est cachée sous la tasse voisine de la tasse où se cache la jaune.
3. La balle violette est cachée sous une tasse de plus petit numéro que la balle orange.
4. La balle rouge est cachée sous la tasse à droite à la tasse de la balle violette.

On souhaite connaître le numéro de la tasse où se cache chaque balle. Écrire le programme CSP à l'aide de l'outil « OR-Tools » qui résout ce problème.

NB: Pour exprimer un « ou logique » entre deux ou plusieurs contraintes, on peut procéder comme dans l'exemple suivant : exprimer la contrainte ($x < 5$ **ou** $x \geq 3$)

```
b=model.NewBoolVar("b")
model.Add(x<5).OnlyEnforceIf(b)
model.Add(x>= 3).OnlyEnforceIf(b.Not())
```

Exercice 06:

4	9	2
3	5	7
8	1	6

Un carré magique normal d'ordre n est un arrangement de n^2 nombres dans un carré, tel que la somme des valeurs sur chaque ligne, chaque colonne chaque diagonale (principale) est égale à une même constante ($n*(n^2+1)/2$) . Un carré magique normal contient les entiers de 1 à n^2 .

Écrire le programme CSP à l'aide de l'outil « OR-Tools » qui résout ce problème pour $n=3$, $n=6$, $n=10$.

Indications :

pour résoudre l'exercice on utilise la modélisation suivante :

Les variables : $X = \{X_{ij} / ij \in [1..n] \}$.

Les domaines : $D(X) = \{1, 2, \dots, n^2\}$.

Les contraintes : On suppose que la somme magique égale à S ($n*(n^2+1)/2$)

- All_different($X_{11}, X_{12}, \dots, X_{nn}$),

- La somme de chaque ligne égale à S : $\forall i \in [1..n] \sum_{j=1}^n X_{ij} = S$

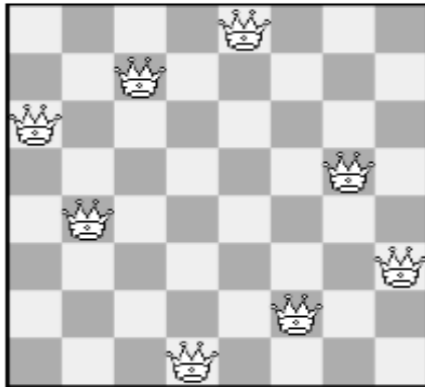
- La somme de chaque colonne égale à S $\forall j \in [1..n] \sum_{i=1}^n X_{ij} = S$ colonnes

- La somme du diagonale principale ($i=j$) égale à S : $\forall i, j \in [1..n] \text{ Si } i=j, \sum X_{ij} = S$

- La somme du diagonale secondaire ($i+j=n+1$) $\forall i, j \in [1..n] \text{ Si } i+j=n+1, \sum X_{ij} = S$

Exercice 07:

le problème de 8-reines consiste à huit reines sur un échiquier (grille 8 x 8) de sorte qu'aucune reine n'en attaque une autre, c'est-à-dire qu'il n'y ait pas deux reines sur la même colonne, la même ligne ou la même diagonale.



1. Écrire le programme CSP à l'aide de l'outil « OR-Tools » qui résout le problème de 8-reines.
2. Combien de solution existe-il ?
3. Pour vérifier visuellement le placement des 8 reines sur l'échiquier, le programme suivant utilise la bibliothèque « matplotlib » pour dessiner un échiquier et y placer un cercle représentant une reine.

```
import matplotlib.pyplot as plt
```

```
fig, ax = plt.subplots(figsize=(8, 8))
```

```
# Draw the chess board
```

```
ax.set_xlim(0, 8)
```

```
ax.set_ylim(0, 8)
```

```
# Color the chess board squares
```

```
for i in range(8):
```

```
    for j in range(8):
```

```
        if (i + j) % 2 == 0:
```

```
            ax.add_patch(plt.Rectangle((j, i), 1, 1, facecolor='#769656'))
```

```
        else:
```

```
            ax.add_patch(plt.Rectangle((j, i), 1, 1, facecolor='#eeeeed2'))
```

```
# Place one queen in the position (5,4)
```

```
ax.add_patch(plt.Circle((5+0.5, 4-0.5), 0.2, facecolor='red', edgecolor='black'))
```

```
# Set the title and show the plot
```

```
ax.set_title('8-Queens Problem Solution')
```

```
plt.show()
```

- Utilisez le programme précédent pour vérifier visuellement l'une des solutions (choisie aléatoirement) retournée par votre programme.

NB : La modélisation de ce problème est faite dans le cours (voir les diapos).

TP N°04 : Programmation par contraintes

Exercice 08 :

Dans un **Sudoku** il y a 81 cases où on peut mettre les chiffres de 1 à 9. On doit mettre les chiffres dans les cases tel que chaque horizontale, verticale ou bloque de 9 cases contienne les chiffres différents.

	3		4	5		7	
6	2			8		4	
7				1			9
2		6			3	8	
						2	3
	1	3	6			9	5
		8		4	7		
							6
		9		5		3	8
				3	8	2	

- Écrire le programme CSP à l'aide de l'outil « OR-Tools » pour résoudre le Sudoku de la figure jointe. On peut représenter ce sudoku sous la forme suivante :

```
puzzle = [
    [0, 3, 0, 4, 0, 5, 0, 7, 0],
    [6, 2, 0, 0, 8, 0, 4, 0, 0],
    [7, 0, 0, 0, 0, 1, 0, 0, 9],
    [2, 0, 6, 0, 0, 3, 8, 0, 0],
    [0, 0, 0, 0, 0, 0, 2, 0, 3],
    [0, 1, 3, 6, 0, 0, 9, 5, 0],
    [0, 0, 8, 0, 4, 7, 0, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 0, 6],
    [0, 0, 9, 0, 5, 0, 3, 8, 2]
]
```

Votre programme doit remplacer les 0 par des chiffre de 1 à 9.

Indications :

Pour résoudre l'exercice on utilise la modélisation suivante :

Les variables : x_{ij} — le chiffre dans le carre (i, j) , avec $ij \in [1..9]$ (ou $ij \in [0..8]$).

Les domaines : $D(x_{ij}) = \{1, \dots, 9\}$, $\forall (i, j)$.

Les contraintes :

- Les chiffres dans chaque ligne sont différents :

$\text{all-different}(x_{i1}, x_{i2}, \dots, x_{i9}), 1 \leq i \leq 9,$

- Les chiffres dans chaque colonne sont différents :

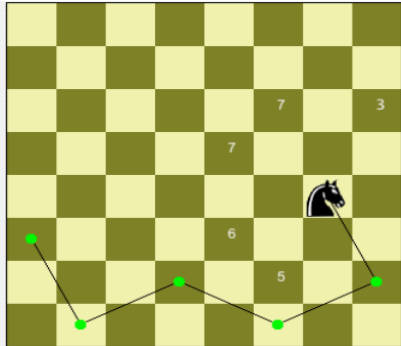
$\text{all-different}(x_{1j}, x_{2j}, \dots, x_{9j}), 1 \leq j \leq 9,$

-Les chiffres dans chaque bloc 3×3 sont différents :

$\text{all-different}(X_{3k+1,3l+1}, X_{3k+1,3l+2}, \dots, X_{3k+3,3l+3}), 0 \leq k, l \leq 2.$

Exercice 09: (Supplémentaire)

Le problème du **cavalier d'Euler** consiste à trouver un chemin permettant à un cavalier de visiter chaque case d'un échiquier, en revenant à son point de départ sans jamais passer deux fois par la même case (voir la figure).



- Écrire le programme CSP à l'aide de l'outil « OR-Tools » qui résout le Sudoku de la figure jointe.

Indications :

Pour résoudre l'exercice on utilise la modélisation suivante :

On va utiliser les variables entières x_i et y_i , $i = 0, \dots, 64$, pour désigner les coordonnées (x_i, y_i) de la case où se trouve le cavalier après le i -ème pas. Le domaine de ces variables est $\{0, 1, \dots, 7\}$. On va également utiliser les variables entières n_i , $i = 0, \dots, 64$, pour désigner le numéro de la case où se trouve le cavalier après le i -ème pas :

$n_i = 8 \cdot x_i + y_i$. Le domaine des variables n_i est $\{0, 1, \dots, 63\}$.

Comme le cavalier ne peut pas repasser sur une même case (sauf pour la case de départ), toutes les valeurs des variables n sauf n_0 doivent être différentes : $\text{all-different}(n_1, n_2, \dots, n_{64})$.

Cavalier doit revenir à son point de départ :

$n_0 = n_{64}$.

- les contraintes de déplacement du cavalier s'expriment comme suit :

$$\begin{cases} |x_i - x_{i+1}| + |y_i - y_{i+1}| = 3, \\ x_i \neq x_{i+1}, \\ y_i \neq y_{i+1}. \end{cases} \quad \forall i = 0, \dots, 63.$$